

Introduction

Sliding window problems show up constantly in coding interviews because they test something more valuable than syntax: can you notice when you're doing repeated work you don't actually need? LeetCode 3471, Find the Largest Almost Missing Integer, looks like a straightforward "scan every window" problem on the surface. The brute force is obvious and easy to write. But it's also a trap — it runs in $O(n \cdot k)$ time, and the moment an interviewer bumps up the constraints, that solution falls over.

What makes this problem worth your time isn't the final code — it's the reasoning that gets you there. You'll practice a skill that transfers to dozens of other sliding window problems: proving that most of your input is irrelevant to the answer, so you never have to touch it. That's the kind of insight interviewers are actually screening for when they hand you a window problem.

Problem Overview

You're given an array of integers `nums` and an integer `k`. Look at every contiguous window of exactly `k` elements in the array. A number is called "almost missing" if it appears in exactly one of those windows — not zero, not two or more, exactly one.

Your task is to return the largest value in the array that qualifies as almost missing. If no value appears in exactly one window, return `-1`.

The subtlety is that a value can appear at multiple positions in the array. Every one of those positions contributes to that value's total window coverage, and you have to add all of them up before deciding whether it's really "almost missing."

Example

Take `nums = [3, 9, 2, 1, 7]` with `k = 3`.

The windows are:

Window Elements

1	[3, 9, 2]
2	[9, 2, 1]
3	[2, 1, 7]

Count how many windows each value appears in:

Value Windows it appears in Count

3	Window 1	1
9	Windows 1, 2	2
2	Windows 1, 2, 3	3
1	Windows 2, 3	2
7	Window 3	1

Only 3 and 7 appear in exactly one window. Between them, 7 is larger, so the answer is 7.

Intuition

Here's how an experienced engineer would approach this rather than just diving into code.

Start with the honest question: what does the brute force actually cost you? For every window, you scan up to k elements to count occurrences. With $n - k + 1$ windows, that's $O(n \cdot k)$ work, and every window overlaps heavily with its neighbor — you're re-doing almost the same count over and over.

The real insight comes from asking a different question: which positions in the array can *ever* be almost missing, regardless of their value? Picture the array split into three zones:

- A **left edge zone**: the first $k - 1$ positions.
- A **big middle zone**: everything else.
- A **right edge zone**: the last $k - 1$ positions.

Any position in the middle zone gets covered by every window that could possibly contain it, and because of how sliding windows overlap, that's always exactly k windows. So a middle element is touched k times minimum — it can only be "almost missing" if $k = 1$, which is a degenerate case handled separately. For any $k > 1$, the middle is dead weight. You never need to check it.

That leaves only the two edges. A position near the start of the array is included in fewer windows because there aren't enough windows early enough to reach it — the same logic applies in mirror at the end. So the number of candidates you actually need to examine collapses from n down to roughly $2k$.

There's one more wrinkle: duplicates. If the same value appears at two different indices, each occurrence contributes its own window coverage. You have to sum coverage across *all* occurrences of a value, not just look at one, or you'll wrongly call a duplicate "almost missing" when its combined coverage is actually two or more.

Brute Force Solution

Idea: For every window, count how many times each value appears across all windows, using a dictionary. At the end, scan the dictionary for values that appear exactly once, and take the max.

Algorithm: 1. For each starting index i from 0 to $n - k$, slice out the window `nums[i:i+k]`. 2. For each value in that window, increment its count in a global dictionary. 3. After processing all windows, find the largest key with a count of exactly 1.

Advantages: Simple to write, easy to reason about, hard to get wrong.

Disadvantages: Re-scans overlapping data on every window. Doesn't take advantage of the structural insight that only edge elements matter.

Time complexity: $O(n \cdot k)$ — $n - k + 1$ windows, each costing up to k work. **Space complexity:** $O(n)$ for the counting dictionary in the worst case.

Optimal Solution

The optimal approach never builds or scans windows directly. Instead, it computes each candidate's *total window coverage* using a formula, and only checks candidates that could possibly qualify.

Step 1 — Handle the giant- k case. If $k \geq n$, there's at most one window in existence (or none). Every value is automatically "almost missing" in that case, so the answer is simply the maximum of the array.

Step 2 — Coverage formula for a single index. For an index i (0-indexed) in an array of length n with window size k , the number of windows that include index i is:

$$\text{coverage}(i) = \min(i, k - 1, n - k, n - 1 - i) + 1$$

This works because a window's start position s must satisfy $s \leq i$ and $s \geq i - k + 1$, combined with the valid range $0 \leq s \leq n - k$. The formula folds all four boundary constraints into one expression using whichever limit is tightest — left edge, right edge, or window size.

Step 3 — Collect edge candidates. Only positions in the first $k - 1$ and last $k - 1$ indices can have coverage less than k (and therefore possibly equal to 1). Build this candidate index set instead of scanning the whole array.

Step 4 — Sum coverage per value. For each *value* that appears at one or more candidate indices, sum `coverage(i)` over **every** index where that value occurs anywhere in the array (not just edge

positions) — this is what correctly handles duplicates.

Step 5 — Pick the answer. Among values whose total coverage equals exactly 1, return the largest. If none qualify, return `-1`.

Visually, the candidate zones look like this:

[left edge: k-1 positions]	[middle (skipped)]	[right edge: k-1 positions]
candidates	never checked	candidates

Python Code

```
from typing import List

def largest_almost_missing_integer(nums: List[int], k: int) -> int:
    n = len(nums)

    # Every value is a candidate if k spans the whole array (or more).
    if k >= n:
        return max(nums)

    def window_coverage(index: int) -> int:
        """Number of windows of size k that include this index."""
        return min(index, k - 1, n - k, n - 1 - index) + 1

    # Only the first k-1 and last k-1 positions can ever be almost missing.
    edge_indices = set(range(k - 1)) | set(range(n - k + 1, n))

    # Map each candidate value to every index where it occurs in the array.
    value_to_indices = {}
    for i, val in enumerate(nums):
        value_to_indices.setdefault(val, []).append(i)

    best = -1
    seen_values = {nums[i] for i in edge_indices}

    for val in seen_values:
        total_coverage = sum(window_coverage(i) for i in value_to_indices[val])
        if total_coverage == 1:
            best = max(best, val)

    return best
```

Code Walkthrough

- `if k >= n: return max(nums)` — handles the case where there's effectively one window or none; everything qualifies, so the largest value wins outright.
- `window_coverage(index)` — a single formula replacing the need to actually simulate windows. It takes the tightest of four constraints: how far the index is from the start, from the end, and how `k` limits things on both sides.
- `edge_indices` — a set built from the first `k - 1` and last `k - 1` positions. This is the only part of the array we'll ever inspect for candidacy.
- `value_to_indices` — a dictionary mapping every value to *all* its positions in the array, so that once we decide a value is worth checking, we can sum its true total coverage, duplicates included.
- `seen_values` — the set of distinct values sitting at edge positions; these are the only values that can possibly have total coverage of 1.
- The final loop sums each candidate value's coverage across all its occurrences and keeps the largest value whose total lands on exactly 1.

Dry Run

`nums = [3, 9, 2, 1, 7]` , `k = 3` , `n = 5` .

`k < n` , so we proceed past the giant-`k` check.

`edge_indices = {0, 1} | {3, 4} = {0, 1, 3, 4}` — index `2` (the middle) is skipped entirely.

`seen_values = {3, 9, 1, 7}` (the values sitting at those edge indices).

Compute coverage for each:

Value Indices coverage(i) per index Total

3	[0]	$\min(0, 2, 2, 4)+1 = 1$	1
9	[1]	$\min(1, 2, 2, 3)+1 = 2$	2
1	[3]	$\min(3, 2, 2, 1)+1 = 2$	2
7	[4]	$\min(4, 2, 2, 0)+1 = 1$	1

Values with total coverage exactly 1: `3` and `7` . The larger is `7` , matching the expected output.

Complexity Analysis

Time: $O(n)$. Building `edge_indices` and `value_to_indices` is $O(n)$. The candidate set has at most $2(k-1)$ distinct values, and summing their coverage touches each occurrence of each candidate value at most once across the whole array — bounded by $O(n)$ total work, since no index is processed more than a constant number of times in aggregate.

Space: $O(n)$ for `value_to_indices`, which in the worst case stores every index of the array.

This is correct because the key structural fact — middle elements always have coverage `k`, which can only equal 1 when `k = 1` — lets us discard the vast majority of the array without ever looking at it.

Alternative Solutions

You could also compute coverage using a prefix-sum / difference-array trick: increment a delta array at each window's start and decrement just past its end, then take a running sum to get per-index coverage in one pass, and separately track duplicates. This is mathematically equivalent to the closed-form `window_coverage` formula above but does the work with array increments instead of a `min()` call. Both are $O(n)$; the closed-form version is slightly simpler to read and avoids allocating an extra array, so it's the better default here.

Edge Cases

- `k == 1`: every window is a single element, so every value technically has coverage equal to its own frequency. Values appearing exactly once in the whole array qualify.
- `k == n`: handled explicitly — there's exactly one window (or, if `k > n`, conceptually "the whole array"), so every value is almost missing and the answer is `max(nums)`.
- **All duplicate values:** e.g., `[5, 5, 5, 5]` — coverage sums will almost always exceed 1 unless `k` is large enough to collapse to the giant-`k` case.
- **No qualifying value:** return `-1` — this happens when every value's total coverage is 0 (impossible if it's in the array) or 2+.
- **Negative numbers:** the algorithm doesn't care about sign; comparisons and `max()` work identically.
- **Minimum array size:** `n == 1` with `k == 1` — trivially returns that single element via the giant-`k` path.

Common Mistakes

1. **Forgetting to sum coverage across all occurrences of a duplicate value.** Checking only the edge occurrence and ignoring a middle occurrence of the same value will wrongly mark it as almost missing.
2. **Scanning the whole array for candidates instead of just the edges.** This silently degrades your solution back toward $O(n \cdot k)$ or at least wastes unnecessary work.

3. **Off-by-one errors in the coverage formula.** Mixing up $k - 1$ vs k , or forgetting the $+1$ at the end, produces coverage counts that are systematically wrong by one.
4. **Not special-casing $k \geq n$.** The edge-index logic assumes $k < n$; skipping the guard can cause negative ranges or index errors.
5. **Assuming "largest value" means "largest index."** The problem asks for the largest *value*, not the rightmost qualifying position — easy to swap by accident when writing the final comparison.

Interview Questions

- How would this change if you needed the *smallest* almost missing integer instead of the largest?
- Can you derive the `window_coverage` formula from first principles on a whiteboard, without me giving it to you?
- What if k could be 0? How would you handle that as a precondition?
- How would you adapt this if windows could wrap around the array circularly?
- Can you solve this using a difference array instead of the closed-form formula, and what's the tradeoff?

Similar Problems

- **LeetCode 239 — Sliding Window Maximum:** another problem about extracting information from every window efficiently, using a monotonic deque instead of a coverage formula.
- **LeetCode 1004 — Max Consecutive Ones III:** a two-pointer sliding window problem that also relies on recognizing which parts of the array actually matter.
- **LeetCode 76 — Minimum Window Substring:** teaches the same discipline of avoiding re-scanning overlapping windows, applied to strings.
- **LeetCode 2461 — Maximum Sum of Distinct Subarrays With Length K:** reinforces reasoning about fixed-size windows and per-element contribution.

Key Takeaways

The brute force here isn't wrong, it's just blind — it treats every element as equally worth checking. The optimal solution comes from asking which elements *can't possibly* be the answer and eliminating them entirely, rather than optimizing how you check each one. Once you see that only the edges matter, and that duplicates need their coverage summed rather than checked once, the rest is arithmetic. This

pattern — proving a large chunk of your input is structurally irrelevant — is one of the highest-leverage habits you can build for sliding window problems.

Watch the Video

Prefer to see this solved step by step, with a live dry run and timing comparison against the brute force? Watch the full walkthrough here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series where I work through the LeetCode Daily Challenge in Python, one problem at a time. The goal isn't just to get to a working answer — it's to build the reasoning skills that let you spot the optimal approach on your own, in an interview or otherwise. New problems drop regularly, always with the brute force, the reasoning, and the optimized solution side by side.

Call To Action

If this walkthrough helped the problem click, the video version has the full dry run and a live timing comparison — go like it and subscribe on YouTube so you don't miss the next daily problem. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. And if you've got a cleaner approach or a follow-up question, drop it in the comments — I read all of them.

Quick Review

Trigger: array, need max value that's covered by $\leq k$ of the other windows. **Pattern?**

Sliding window with a count array — check window coverage per index, not per window.

Time complexity of this almost-missing-integer solution?

$O(n \cdot k)$ — for each of n candidate values, scan up to k windows containing it.

Space complexity?

$O(1)$ extra beyond input — just a few counters and the answer variable, no auxiliary array needed.

Trickiest bug: which indices can even be 'almost missing'?

Only elements near the array edges (first $k-1$ and last $k-1$) can appear in $<k$ windows; middle ones can't qualify.

Brute force vs edge-only scan — why does brute force struggle?

Brute force checks every window for every value ($O(n^2)$); edge-only exploits that interior values always hit all k windows, so skip them.

Follow-up: how would duplicates of the target value affect the answer?

If a value repeats, it's 'almost missing' only if every occurrence individually is covered by $<k$ windows.

Find the Largest Almost Missing Integer — free Python cheat sheet